Provisioning AWS Resources Using Ansible

Automating Cloud Deployment with Ansible

Introduction

Provisioning cloud resources manually on AWS can be **time-consuming and prone to errors**. **Ansible automates the provisioning of AWS resources** such as EC2 instances, S3 buckets, RDS databases, and Security Groups using simple and reusable Playbooks.

This guide will cover:

- **Why use Ansible for AWS provisioning?**
- **Setting up Ansible for AWS**
- **Creating AWS resources with Ansible Playbooks**
- **Best practices and troubleshooting**

Why Use Ansible for AWS Provisioning?

Feature	Benefit
Agentless	No need to install additional software on AWS instances.
Infrastructure as Code (IaC)	Define AWS infrastructure in Ansible Playbooks.
Security	Uses IAM roles and AWS API securely.
Scalability	Manage multiple AWS resources at once.
Multi-Cloud Support	Works across AWS, Azure, and GCP.

💡 *Ansible enables* **fast, repeatable, and error-free AWS deployments***!*

Setting Up Ansible for AWS

Before provisioning AWS resources, **install Ansible and configure AWS access** on your system.

Step 1: Install Ansible and AWS SDK

```
sudo apt update
sudo apt install -y ansible
pip install boto boto3 botocore
```

Step 2: Configure AWS Credentials

Create a credentials file:

```
mkdir -p ~/.aws
nano ~/.aws/credentials
```

Add the following:

```
[default]
aws_access_key_id = YOUR_ACCESS_KEY
aws_secret_access_key = YOUR_SECRET_KEY
region = us-east-1
```

Verify AWS access:

```
aws s3 ls
```

💡 *Ensure your IAM user has the necessary permissions to create AWS resources!*

Provisioning AWS Resources Using Ansible Playbooks

Once Ansible is configured for AWS, you can use Playbooks to **provision cloud infrastructure automatically**.

1. Provisioning an AWS EC2 Instance

Step 1: Create an Ansible Playbook (launch-ec2.yml)

```
---
- name: Launch an AWS EC2 Instance
 hosts: localhost
 gather_facts: no
 tasks:
   - name: Create EC2 instance
     amazon.aws.ec2_instance:
       name: "MyAnsibleEC2"
       key_name: "my-key"
       instance_type: "t2.micro"
       image_id: "ami-12345678"
       region: "us-east-1"
       count: 1
       network:
         assign_public_ip: true
     register: ec2_info

   - name: Display EC2 Instance Details
     debug:
       msg: "Instance ID: {{ ec2_info.instances[0].instance_id }}, Public IP: {{
```

```
ec2_info.instances[0].public_ip_address }}"
```

Step 2: Run the Playbook

```
ansible-playbook launch-ec2.yml
```

💡 *This Playbook creates an* **AWS EC2 instance** *and prints its* **instance ID and public IP***!*

---

2. Provisioning an AWS S3 Bucket

Step 1: Create an Ansible Playbook (create-s3.yml)

```
---
- name: Create an AWS S3 Bucket
 hosts: localhost
 gather_facts: no
 tasks:
   - name: Create an S3 Bucket
     amazon.aws.s3_bucket:
       name: "my-ansible-bucket"
       state: present
       region: us-east-1
```

Step 2: Run the Playbook

```
ansible-playbook create-s3.yml
```

💡 *This Playbook provisions an* **AWS S3 bucket** *to store files securely.*

3. Provisioning an AWS RDS Database

Step 1: Create an Ansible Playbook (create-rds.yml)

```
---
- name: Create an AWS RDS Instance
 hosts: localhost
 gather_facts: no
 tasks:
  - name: Launch RDS instance
    amazon.aws.rds_instance:
      db_instance_identifier: my-ansible-db
      engine: mysql
      db_instance_class: db.t3.micro
      allocated_storage: 20
      master_username: admin
      master_user_password: password123
      region: us-east-1
      publicly_accessible: yes
      state: present
```

Step 2: Run the Playbook

```
ansible-playbook create-rds.yml
```

💡 *This Playbook provisions an* **AWS RDS MySQL database**.

4. Provisioning an AWS Security Group

Step 1: Create an Ansible Playbook (security-group.yml)

```
---
- name: Create an AWS Security Group
 hosts: localhost
 gather_facts: no
 tasks:
  - name: Create Security Group
    amazon.aws.ec2_security_group:
      name: allow-web-traffic
      description: "Allow HTTP and SSH access"
      region: us-east-1
      rules:
        - proto: tcp
          ports:
            - 22
            - 80
          cidr_ip: 0.0.0.0/0
```

Step 2: Run the Playbook

```
ansible-playbook security-group.yml
```

💡 *This Playbook provisions an* **AWS Security Group allowing SSH and HTTP traffic***!*

Best Practices for Provisioning AWS with Ansible

1. **Use IAM Roles Instead of Static Keys** – Assign AWS IAM roles for security.
2. **Store Variables Securely** – Use **Ansible Vault** for sensitive data.
3. **Use Dynamic Inventory for AWS** – Fetch AWS instance lists dynamically.
4. **Enable Logging and Debugging** – Use -vvv for troubleshooting.
5. **Automate with CI/CD Pipelines** – Integrate with **Jenkins, GitHub Actions, or GitLab CI/CD**.

💡 *Following best practices ensures* **secure and scalable AWS automation***!*

Common Issues and Troubleshooting

Problem	Solution
botocore.exceptions.NoCredentialsError	Ensure AWS credentials are configured correctly.
UnauthorizedOperation	Check IAM role permissions for EC2, S3, or IAM.
Instance not found	Ensure the correct **region and instance ID** are used.
Connection timeout	Verify **security group and firewall settings**.

Know More (FAQs)

1. Can I configure multiple AWS regions?

Yes! Modify aws_ec2.yml to include multiple regions:

```
regions:
 - us-east-1
 - us-west-2
```

2. How do I fetch details of all running EC2 instances?

Run:

```
ansible-inventory --list -i aws_ec2.yml
```

3. Can I automate AWS database configuration?

Yes! Use **Ansible RDS modules** to create AWS RDS databases.

4. How do I delete an AWS resource using Ansible?

Change state: absent. Example:

```
tasks:
 - name: Delete an S3 bucket
   amazon.aws.s3_bucket:
     name: my-ansible-bucket
     state: absent
```

5. Can I integrate Ansible with Terraform for AWS?

Yes! Use **Terraform for provisioning** and **Ansible for configuration management**.

Conclusion

Ansible **simplifies AWS provisioning**, allowing IT teams to **automate cloud deployments efficiently**. By automating AWS services like **EC2, S3, RDS, and Security Groups**, businesses can **reduce manual effort and improve security**.

👉 Try provisioning **AWS infrastructure** using Ansible today!